

The Film Is Not a Context Window: Feature-Length Generation as Hierarchical Assembly on a Native Inference Stack

Hanzo AI — ML Systems

Long-form-media crossover in the native-inference series — an inference-systems reading of
feature-length generation

Abstract

A three-hour film is routinely mis-stated as a context-length problem — as though generating it required a model that holds ten thousand seconds of pixels in one attention window. No model does, and our thesis is that none needs to. A feature is not a monolith; it is an *assembly* of roughly two thousand shots of three to eight seconds each [23], and coherence across it decomposes *hierarchically*: the state that must persist across the whole runtime is small and structured — a screenplay and a *film bible* (who the characters are, what the world looks like, what the style is) of tens of thousands of tokens — while the state that is expensive to compute (pixels, waveforms) never needs to persist beyond a single shot, because at every cut the image resets and only the semantics carry over. Long-form generation is therefore not a long-context problem but an *orchestration-and-conditioning* problem layered over *short-horizon* generators, and the long-horizon reasoning that keeps three hours coherent is done in the one place it is cheap: an LLM holding the whole screenplay in a single ordinary context. We develop this in four moves. First, the **coherence stack**: five layers — story, entity, shot, global-style, and audio — each with a persistent object, a conditioning mechanism, a coherence horizon, and a characteristic failure mode, formalized so that each failure has a named cure at its own layer. The load-bearing observation is that *film grammar makes the cut a free reset*: the hard chunk-autoregressive drift problem that the recent forcing literature [7, 8, 9, 10, 11, 12] fights over minute-long clips is, in assembled film, bounded to the length of a single shot — seconds, not hours — and the games companion’s persistence-beyond-the-horizon problem [1] is dissolved here rather than solved. Second, **error accumulation**: we separate intra-shot drift (real, bounded by shot length, measured by the cited video work) from cross-shot identity and style drift (the film-level risk), and propose — marking each proposed unless measured — cross-shot identity similarity and style-consistency metrics, anchored to the one identity axis the series has already measured, the localization dub’s speaker-identity cosine 1.0 [1]. Third, the **systems** reading: a feature render is an *embarrassingly parallel throughput job* — ~ 2000 independent shot-jobs over the series’ measured heterogeneous ring [2] — with the bible as a shared conditioning *prefix* that prefix-caches across every shot, and each shot an *idempotent*, content-hashed unit that makes a three-hour render a resumable build DAG. The *assembly* half of this is now measured end to end in the native orchestrator — a full three-hour placeholder cut assembles in under 30 s on one box (≈ 9 s across three nodes; per-shot ≈ 0.21 – 0.26 s), and re-rendering recomputes *zero* shots — while the *generative* per-shot time stays labelled a projection until the Wan-class video port lands, at which point the same wall-clock template resolves. Fourth, **open problems** specific to film that neither companion owns: perceptual metrics for narrative coherence, the shot-length-versus-drift trade, motif persistence across musical cues, lip-sync integration for dialogue shots, and the rights and provenance of persistent generated identities. The substrate numbers we lean on are measured and cited to the hardware that produced them; every number about a feature-length artifact on our stack is labelled projection.

1 Thesis: no model holds three hours, and none needs to

The reflexive framing of feature-length generation is a scaling framing: today’s open video models generate clips of a few seconds [8, 16], a film is ten thousand seconds, so — the framing goes — the task is to grow the context, the memory, and the coherence horizon of a single generator by three or four orders of magnitude until it can hold an entire film at once. That framing is wrong, and its wrongness is the starting point of this paper. It mistakes the artifact for the computation. A film is not a single ten-thousand-second signal that a viewer experiences as one continuous shot; it is a *cut* sequence of short shots, and the cutting is not incidental — it is the medium’s fundamental grammar.

What a feature actually is, quantitatively. Empirical cinematics settles the structure. Across seventy years of Hollywood film, the average shot length (ASL) fell from roughly twelve seconds in the 1930s to about four in recent decades, and the sequence of shot lengths has a nested waxing-and-waning structure at scales from seconds to tens of minutes [23]. Take a three-hour feature: 180 minutes is 10,800 seconds, and at a contemporary ASL of five to six seconds that is on the order of $N_{\text{shots}} \approx 2000$ shots; an action film at an ASL of two to three seconds has more. The unit of the medium is the *shot*: three to eight seconds of continuous imagery, terminated by a cut. Nothing in a film is longer than a shot except by the viewer’s assembly of shots into scenes and scenes into a story — and that assembly is *semantic*, carried by continuity of character, place, and narrative, not by continuity of pixels.

The state that must persist is small and structured. Ask what actually has to stay consistent across three hours. Not the pixels: shot 1873 shares no pixel with shot 12, and demanding it should would be a category error about how film works. What must stay consistent is a *bounded, symbolic* object — the protagonist’s face and wardrobe, the look of a location, the visual style, the through-line of the plot, a character’s voice. A working screenwriter encodes essentially all of it in a screenplay of roughly one page per minute of screen time: a three-hour film is a ~ 180 -page screenplay, on the order of 40,000 tokens, plus a *film bible* of character and location descriptions and reference images. That is the entire long-horizon state, and it is *tiny*: 40,000 tokens fits, with enormous margin, inside a single present-day LLM context of hundreds of thousands to millions of tokens. The long-horizon problem, stated correctly, is already solved — not by a bigger video model, but by moving the long-horizon reasoning into the *text* domain, where three hours of narrative is a short document.

The thesis. Long-form generative media is an *orchestration and conditioning* problem over *short-horizon* generators, not a long-context generation problem. A single LLM holds the whole film’s structured plan in one context and is the long-horizon engine; a fleet of short-horizon generators (a few-second video model, a per-line speech model, a per-cue music model, an image model) renders the leaves; and coherence is manufactured *hierarchically*, by conditioning every leaf on the small persistent state the plan defines and re-injecting it shot by shot. This inverts the scaling framing: the hard part is not holding three hours in a network, it is the *decomposition* — authoring the hierarchy, propagating identity and style down it, and *assembling* the leaves back into a timeline — and the decomposition is an inference-systems problem our native stack is well shaped to serve, because every leaf is a model inference and the orchestration is a throughput schedule over them.

Relation to the companions, stated up front. This paper sits deliberately between two others and duplicates neither. The games companion [1] owns the *within-shot* computation: a shot is a short video clip, generated by exactly the fixed-shape frame loop — “the frame is the token” — whose latency budget, two-regime partition (launch-bound denoise loop versus compute-bound VAE), and cures that paper establishes; we take that analysis as given and never re-derive it, and we cross-reference it wherever a shot’s internal cost is at issue. The distributed companions own the *substrate*: the measured cross-vendor ring [2], the latency physics of decode [3], and the eight open problems of heterogeneous distributed inference [4]. What is *new here* is the layer *above* the shot and *above* the ring: the hierarchical-assembly structure of a feature, its coherence stack, its error budget, and the throughput schedule that renders it. One sharp difference from the games companion orients everything below. There, the artifact is real-time and interactive, a single continuous frame loop under a 33–66 ms deadline, and *persistence beyond the KV horizon is the central open problem* — a room you looked away from must still be there when you look back. Here, the artifact is offline and *pre-planned*: there is no per-frame deadline (a render is a batch job), the whole film exists as a plan before a single pixel is drawn, and the cut means we *never* need continuity longer than one shot. The games companion’s persistence problem is not solved in this paper; it is *dissolved* by film grammar, and saying exactly how is the content of §2.

2 The coherence stack

Coherence in a feature is not one property; it is five, at five scales, each maintained by a different mechanism and each failing in a different way. The engineering discipline of long-form generation is to name the layers, give each its own persistent object and conditioning mechanism, and fix each layer’s failure at that layer rather than reaching for a single monolithic model to hold all five at once. Table 1 is the whole section in one view; the subsections develop each row. Throughout, the persistent objects are *data* — JSON plans, embeddings, reference images, LoRA adapters, voice ids — not model weights, and every conditioning mechanism is an *input* to a short-horizon generator, which is why the stack composes on inference infrastructure the series already has.

2.1 Story layer: the LLM is the long-horizon engine

Object and mechanism. The top of the stack is a *hierarchical plan*: logline → acts → scenes → shots, authored by an LLM and materialized as a tree of structured records. A shot record names its location, the characters present, the camera and action, the dialogue lines, the intended duration, and references (by id) into the entity and style layers below. Because the whole plan is ~40k tokens (§1), a single LLM holds it in one context and authors it *globally*: the model that writes shot 1873 has shot 12 in view, so setups pay off, arcs close, and the plot is consistent by construction — the LLM *is* the long-horizon engine, and its context window, not any video model’s, is where three hours of narrative coherence actually lives. This is the technical core of the thesis: the coherence that the scaling framing wanted to buy with a giant video context is bought instead with an ordinary text context, because narrative is symbolic and compresses to tens of thousands of tokens.

Failure mode: plan drift. The characteristic failure is *plan drift* — an LLM emitting a plan that is locally plausible but globally inconsistent (a character who dies in scene 40 speaks in scene 47; a shot that references a location id never defined; a scene whose shot durations do not sum to its intended length). Free-form generation makes this failure invisible until render time.

Layer	Persistent object	Conditioning mechanism	Coherence horizon	Failure mode	→
Story	screenplay / scene / shot tree (~40k tok)	LLM context (whole plan in one window)	whole film	plan drift → structured-output schema + validation	→
Entity	film bible: id embeddings, LoRA, voice ids	re-inject reference into every shot's conditioning	whole film	identity drift → embedding-anchored re-injection	→
Shot	tail frames / KV of the current clip	chunk-autoregressive tail-frame conditioning	one shot (s)	intra-shot drift → short horizon; forcing/anti-drift	→
Global style	style prompt fragment + grade LUT	shared style token + assembly-time color grade	whole film	style flicker → shared conditioning + one grade pass	→
Audio	voice ids, motifs, stems	per-line TTS, per-cue score, per-shot Foley; mix	whole film	audio discontinuity → timeline assembly, not generation	→

Table 1: The coherence stack. Each layer maintains one scale of consistency with its own persistent object (all *data*, not weights) and its own conditioning mechanism, and fails in its own way with a cure local to the layer. The horizons are the key: only the *shot* layer has a short (seconds) horizon, and it is the only layer where the hard chunk-autoregressive drift problem lives; every other layer’s persistence is carried by a small structured object re-injected as conditioning, not by a long generative context. This is the whole argument of §1 made operational.

Cure: structured-output schemas and validation. The cure is to make the plan a *typed* object and check it before rendering. Each layer of the tree has a schema (a scene has a location-id that must resolve in the bible; a shot’s character-ids must be a subset of its scene’s; durations must sum; every referenced style and voice id must exist), and the LLM emits against the schema under constrained decoding, with a validation pass that rejects or repairs a plan that violates a cross-reference. This is the film analog of the series’ broader discipline of validating at boundaries: the plan is the boundary between the long-horizon text engine and the short-horizon renderers, and a malformed plan is a bug caught cheaply in the text domain rather than expensively after two thousand shots have rendered. The plan is also what makes the render a *DAG* (§4): a validated, content-addressable description of every leaf and its inputs.

2.2 Entity layer: the film bible is memory-as-retrieval

Object and mechanism. The entity layer is the *film bible*: for each recurring character, a stable identity representation — one or more reference images, an identity embedding, and, where fidelity demands it, a per-character low-rank adapter (LoRA) [19, 20] — and a voice id; for each location, a reference and description; for props and creatures likewise. The mechanism is *re-injection*: every shot that features a character conditions its generator on that character’s identity representation, via image-prompt adaptation [21], a loaded LoRA, or reference-image conditioning, so the character is rendered from the *same* anchor in shot 12 and shot 1873. Identity is thus maintained not by the generator *remembering* the character across two thousand shots — it remembers nothing between independent shot-jobs — but by the orchestrator *retrieving* the identity from the bible and re-supplying it each time.

This is the film analog of position-keyed world memory. The games companion identifies, as an open problem for interactive world models, an external addressable *world-state memory* that re-conditions the frame loop when a region re-enters view, so consistency is bounded by the store’s fidelity rather than by the KV window [1]. The film bible is the same idea in a setting where it is not open but *routine*: an external, addressable store of “what everything is,” read on every shot and written once at authoring time, whose retrieval key is the character/location id rather than a spatial position. The reason it is routine here and open there is exactly the pre-planning difference of §1: an interactive world model does not know in advance which region will re-enter the window and must discover and store it online, whereas a film’s plan names every entity in every shot *before* rendering, so retrieval is a static lookup, not an online memory system. Memory-as-retrieval is the cure; the plan is what makes the retrieval trivial.

Failure mode and cure. The failure is *identity drift*: the same character rendered subtly (or grossly) differently across shots — face, age, wardrobe, or, for a voice, timbre. The cure is embedding-anchored re-injection with enough conditioning strength that the identity is pinned, and, where a bare reference embedding is too weak (fine facial identity, an unusual creature), a per-character LoRA that has memorized the identity into weights that are loaded for that character’s shots. The measurement axis is cross-shot identity similarity (§3); on the *audio* side this axis is already measured for our stack, at speaker-identity cosine 1.0 for the native localization dub [1], which is the existence proof that re-injected identity conditioning can hold a character constant across independently generated segments. The visual analog is proposed, not yet measured.

2.3 Shot layer: where the only hard drift problem lives — and how film grammar bounds it

Object and mechanism. A shot is a three-to-eight-second clip, generated by the frame-loop machinery the games companion analyzes [1]; within a shot, temporal continuity (a hand stays a hand, motion is smooth) is maintained by *chunk-autoregressive* conditioning: the model generates a chunk of frames, then conditions the next chunk on the tail frames (and the KV) of the last, extending the clip while carrying visual state forward. This is the mechanism the recent literature has converged on for extending short generators — diffusion forcing and its descendants condition each frame or chunk on previously generated context [7, 8], and the distillation and caching variants make it real-time and stable [9, 10, 11]. The shot layer is the *only* layer whose persistent object is a generative context (tail frames, KV) rather than a small symbolic anchor, and therefore the only layer where chunk-autoregressive drift — the accumulation of error as each chunk conditions on the last — is a first-order concern.

The key observation: the cut is a free reset. Here is where film grammar pays off as a systems property. The chunk-autoregressive literature fights drift over *minute-scale* clips: TetherCache exists to stabilize “long-form” autoregressive video against “accumulated distribution drift” [11], FramePack packs frame context and prevents drift to reach “thousands of frames” [12], RAD adds a global memory to combat forgetting over long video [13], and WorldPlay engineers long-term geometric consistency for extended interactive sequences [14]. All of that difficulty is the difficulty of holding one continuous shot coherent for a long time. *In assembled film, we do not need a long continuous shot.* At every cut the image resets completely — the next shot shares no pixel with the previous one, only semantics (the same character, now from a different angle; the same location, later) — and the semantics are carried by the entity and style layers as re-injected conditioning, not by tail-frame continuity. So the chunk-autoregressive drift problem is bounded to the length

of a single shot: seconds, not minutes, and certainly not hours. Feature-length coherence does *not* require solving long-horizon video generation; it requires generating a three-to-eight-second clip well and cutting. The forcing literature we cite is exactly the right tool at exactly the right scale — a shot — and the assembly structure means we never pay for its hardest regime.

Failure mode and cure. The failure is intra-shot drift, and its cure is threefold and all at the shot layer: keep the horizon short (a shot is short by grammar, so the accumulation has little time to compound); use the anti-drift conditioning the cited work provides [11, 12] within the shot; and, when a shot must be long, prefer a cut — the director’s instrument is also the engineer’s. This is the one place the games companion’s within-shot analysis and this paper’s cross-shot analysis meet: a shot’s *internal* budget and drift are the games companion’s subject, and the *number* of shots and their *assembly* are this paper’s.

2.4 Global-style layer: shared conditioning and a single grade

Object and mechanism. A film has a look — palette, contrast, grain, lens character, an overall visual grammar — that must be constant across all two thousand shots even though they are rendered independently, possibly on different nodes at different times. The persistent object is a style specification: a style prompt fragment and, optionally, a style LoRA and a reference frame, plus a color-grade transform (a LUT). The mechanism is two-part: *shared style conditioning* injected into every shot’s generation so the shots are drawn in the same style to begin with, and an *assembly-time grade* — a single deterministic color transform applied to every rendered shot in the final timeline — that removes residual shot-to-shot variation the way a colorist unifies footage from different cameras. Style is thus enforced twice: softly at generation and exactly at assembly.

Failure mode and cure. The failure is *style flicker*: perceptible jumps in look across cuts, the tell of independently generated shots. The cure is that the grade is a *pure function* applied uniformly, so it cannot itself introduce variation, and the shared conditioning narrows the gap the grade must close. Style consistency is the second proposed measurement axis of §3. This layer is also the clearest small illustration of the paper’s method: a whole-film property (constant look) maintained not by a model that holds the whole film, but by a small shared object (a style spec, a LUT) re-applied to every independently generated leaf.

2.5 Audio: long-form audio is assembly, not generation

The audio stack mirrors the visual one, and is if anything cleaner. A feature’s soundtrack is three superimposed layers, each generated at its natural short unit and *assembled* on a timeline, so that “three hours of audio” is, like three hours of video, never generated as one signal.

- **Dialogue is per-line TTS.** Each spoken line is a short generation, conditioned on the speaking character’s voice id from the bible (§2.2); speech is natively unbounded because a film has no single utterance longer than a sentence or two, and voice identity is held across the film by the same re-injection as visual identity — the axis measured at cosine 1.0 [1]. Dialogue audio is a per-line job, placed at the line’s timecode.
- **Score is per-cue, with shared motif conditioning.** A film score is not one long piece but a set of *cues*, each a few tens of seconds under a scene, and this is how human composers work: a small set of *motifs* (a character’s theme, the main title) recur, re-orchestrated per

cue. Generative music at cue scale [22] conditioned on a shared motif is the mechanism; per-scene cues placed on the timeline are the assembly. Motif *persistence* across cues — generating recognizably the same theme in different cues — is genuinely open and we name it so (§5).

- **Foley and ambience are per-shot.** Diegetic sound (footsteps, doors, room tone) is short and local to a shot or scene, generated or retrieved per shot and placed.

The mechanism is a mixdown, and the failure is a mix failure. The three layers plus any effects are combined by *timeline mixing* — the deterministic sum of stems at their timecodes with levels and crossfades — exactly as a dub is mixed. Long-form audio coherence is therefore an *assembly* property, not a generation property: nothing generates three hours of audio; the orchestrator generates thousands of short audio units and mixes them at their times. The failure mode is discontinuity at a boundary (a hard cut in room tone, a level jump between cues), and the cure is the assembly craft that solves it in real post-production — crossfades, consistent ambience beds, level automation — applied deterministically at mix time, not a smarter generator. This is the audio statement of the whole paper: at every layer, “long” is achieved by assembling “short,” and the intelligence is in the plan and the conditioning, not in a generator that holds the length.

3 Error accumulation: intra-shot drift versus cross-shot drift

The scaling framing’s strongest instinct is that error *accumulates*: chain enough conditioned generations and quality decays, identity wanders, style flickers. The instinct is correct about one kind of chaining and wrong about another, and separating the two is the error budget of long-form assembly.

Intra-shot accumulation is real, and bounded by the shot. Within a shot, chunk-autoregressive generation genuinely accumulates error: each chunk conditions on the last, so quantization, sampling, and modelling error compound frame over frame, which is precisely the drift the forcing and anti-drift literature measures and fights [8, 11, 12, 13]. We do not re-measure it; we cite it, and we observe that assembly *caps its horizon*. Drift that grows with clip length is fought, in continuous-video systems, by ever-cleverer memory and caching to push the tolerable length out to minutes; in assembled film it is capped at the shot length by grammar (§2.3), so the operating point sits in the easy regime of the same curves. The cross-shot boundary is not a site of accumulation at all: because the image resets at the cut, intra-shot error does *not* propagate across it — shot $k+1$ does not inherit shot k ’s accumulated pixel error, only the bible’s (constant) conditioning. The cut is an error *firewall* as well as a free reset.

Cross-shot accumulation is a different quantity: identity and style variance, not decay. What *can* drift across a film is not pixel quality but the *identity* and *style* anchors: a character rendered from the bible in two thousand independent shots may vary shot to shot, and while this is not the compounding decay of a chain (each shot is anchored to the *same* bible reference, not to its predecessor, so there is no feedback), it is a *variance* around the anchor that a viewer reads as inconsistency. The right model of cross-shot error is thus statistical dispersion around a fixed reference, not a random walk that wanders further with every step — a crucial distinction, because dispersion is bounded by conditioning strength and does not grow with N_{shots} , whereas a random walk would. Embedding-anchored re-injection (§2.2) is what makes it dispersion rather than a walk: every shot pulls back to the same anchor.

Proposed metrics (proposed, not measured). To make the cross-shot budget measurable we propose two metrics, and mark both proposed because we have not run them on a feature-length render:

- **Cross-shot identity similarity.** For each recurring character c , embed the character’s appearance in every shot s that features it with a fixed identity encoder ϕ (a face-recognition embedding for humans), and report both the mean similarity to the bible reference, $\frac{1}{|S_c|} \sum_{s \in S_c} \cos(\phi(s), \phi(\text{ref}_c))$, and the dispersion of $\{\phi(s)\}_{s \in S_c}$. High mean and low dispersion is a held identity; the audio instance of exactly this metric is measured at cosine 1.0 for voice [1], which both validates the metric’s form and sets the target the visual axis aims at.
- **Style consistency.** Embed each rendered shot in a style space (a CLIP-image or dedicated style embedding) and report the across-shot variance, before and after the assembly grade (§2.4); the grade should reduce it, and the residual is the style-flicker risk.

Both are *cheap post-hoc measurements over the rendered shots*, not new training objectives, which fits the assembly architecture: the orchestrator already has every shot as a discrete artifact, so scoring the film’s consistency is a pass over the shot set. A first, *structural* version of this scoring — a shot-to-shot luma-cosine proxy — is shipped and measured in the orchestrator [6]; the embedding-based variants (the identity encoder ϕ above, a CLIP-image style embedding) are wired but await a served vision-embedding model, so those remain proposed. Narrative coherence — whether the assembled film makes *sense*, not merely whether its characters look constant — is not captured by either metric and is the first open problem (§5).

4 Systems: a feature render is an embarrassingly parallel throughput job

The assembly thesis has a direct systems consequence: once the plan is authored, the shots are *independent*. Shot k ’s generation depends on the bible and the shot record, not on shot $k-1$ ’s pixels (that is the cut, §2.3), so the two thousand shot-jobs of a feature have *no data dependence among them* and can be rendered in any order, on any number of nodes, concurrently. A feature render is therefore an *embarrassingly parallel throughput job*, and it maps onto the series’ measured heterogeneous substrate cleanly — but by a different decomposition than the games companion’s.

Data-parallel over shots, not batched over viewers. The games companion places interactive play in the throughput regime when there are many *viewers* of one world, batching their rollouts [1]. A feature render is throughput-bound for a different reason: one artifact, many independent *shots*. The natural parallelism is *data-parallel over shots* — each node pulls the next unrendered shot-job, renders it whole, writes the result, and pulls another — a work-queue over a pool of heterogeneous nodes, each running the full short-horizon generator stack locally. Pipeline- and tensor-parallel splitting of a *single* large generator across nodes (the mechanism the distributed companions analyze for a model too big for one node [2, 3]) is the *inner* option, used only when one shot’s model does not fit on one node; the *outer*, dominant parallelism of a film render is shots across nodes, which needs no model splitting at all and so pays none of the sharding or ring-latency costs. The measured substrate we inherit is the cross-vendor fleet itself — an AMD gfx1151 node and an NVIDIA GB10 node decoding coherently over commodity 2.5 GbE at 50.4 tok/s with a canonical-f32 wire [2] — and for a shot-parallel job the wire carries only job descriptions and finished shots, not per-token activations, so the ring’s synchronization tax does not fall on the film’s critical path.

The bible is a shared prefix; the render is prefix-cache reuse at job scale. Every shot conditions on the same bible fragments — the style spec, and, for a given character, that character’s identity conditioning — so across the shot-set there is a large *shared conditioning prefix* and a small per-shot *suffix* (this shot’s action, camera, dialogue). This is prefix caching [3], read at the scale of a render job rather than an interactive session: a node that has rendered one shot of a character holds that character’s conditioning prefix warm and reuses it for the next shot of the same character, recomputing only the divergent suffix. The scheduling consequence is a locality objective — routing a character’s (or a location’s, or a style’s) shots preferentially to the node that already holds that prefix warm turns a cache into a placement heuristic — and it is the batch-throughput dual of the games companion’s interactive edit-loop prefix reuse [1]: same mechanism, different regime (a job over a fixed shot-set, not a live edit stream).

Idempotent, content-hashed shots make the render a resumable build DAG. Each shot is a *pure function* of its inputs: (bible entities it references, shot record, model id, seed) $\mapsto$ shot content. Hash those inputs to a *shot key*; the render is then a build system over N_{shots} targets. Three properties follow, and they are the reason the orchestration state is plain files, not a database. *Resumability*: a render interrupted after 1200 of 2000 shots resumes by re-checking keys and skipping the 1200 whose content already exists — a three-hour render is not an all-or-nothing job. *Incremental re-render*: an edit to the plan (rewrite one scene, recast one character) changes the keys of only the affected shots, so only those re-render, exactly as a build system rebuilds only what a source change invalidates — the film analog of the games companion’s incremental authoring. *Determinism and provenance*: a shot is reproducible from its key (seed included), so the same key yields the same shot across nodes and across time, which both makes the render cacheable and gives every shot a recorded lineage (which model, which weights, which prompt) — the provenance the open problem of §5 builds on. Idempotency is what upgrades “run two thousand jobs” into “evaluate a build DAG,” with all the resumability and caching that implies. The orchestrator confirms this on real hardware: re-running an already-rendered film recomputes *zero* shots, every content key resolving to an existing artifact [6].

Projection: wall-clock on one versus M nodes. Because the shots are independent, the wall-clock model is elementary. Let N_{shots} be the shot count, $\bar{T}_{\text{shot}}$ the mean per-shot render time, and T_{asm} the assembly time (timeline mix and grade). On one node, serial:

$$T_{\text{film}}^{(1)} = N_{\text{shots}} \cdot \bar{T}_{\text{shot}} + T_{\text{asm}}. \quad (1)$$

On M homogeneous nodes rendering shots data-parallel from a shared queue:

$$T_{\text{film}}^{(M)} \approx \left\lceil \frac{N_{\text{shots}}}{M} \right\rceil \cdot \bar{T}_{\text{shot}} + T_{\text{asm}}, \quad \text{speedup } S(M) = \frac{T_{\text{film}}^{(1)}}{T_{\text{film}}^{(M)}} \xrightarrow{T_{\text{asm}} \ll N_{\text{shots}} \bar{T}_{\text{shot}}} \min(M, N_{\text{shots}}), \quad (2)$$

i.e. *near-linear* speedup up to $M = N_{\text{shots}}$ nodes, capped only by the serial assembly tail T_{asm} (Amdahl), which for a film is small: the mix and grade are a single pass over already-rendered material, cheap next to $N_{\text{shots}} \cdot \bar{T}_{\text{shot}}$ of generation. On a *heterogeneous* fleet, replace the even split by the work-queue’s natural load-balancing — fast nodes pull more shots — so the makespan is $\approx N_{\text{shots}} \bar{T}_{\text{shot}} / \sum_j \rho_j$ for node throughputs ρ_j (in shots/s), the shot-parallel analog of the layer-assignment balance the distributed companion optimizes for a split model [4], but without the contiguity constraint because shots are interchangeable. *The assembly half is now measured end to end; only the generative $\bar{T}_{\text{shot}}$ is a projection.* The native film orchestrator (**hanzo-film**, merged

into the engine [6]) runs the full plan-to-timeline pipeline, and on a 20-lane CPU box (node *evo*) with *placeholder* image-sequence shots standing in for the video generator, a shot assembles in $\approx 0.206\text{--}0.256\text{ s}$ end to end — keyframe 9 ms, Ken-Burns motion 123 ms, tail-frame handoff 21 ms, mix 15 ms, mux 63 ms — so a full three-hour placeholder cut ($N_{\text{shots}} \approx 2000$) assembles in under 30 s on one box and $\approx 9\text{ s}$ across three nodes, and an end-to-end mini-film (two scenes, six shots, two recurring characters) renders to a 16.02 s H.264+AAC file (measured by *ffprobe*) [6]. These numbers validate the *assembly* half of Eqs. (1)–(2) on real hardware: the orchestration, prefix reuse, mix, and mux are measured, the near-linear speedup from one box to three appears as predicted, and the serial assembly tail T_{asm} is confirmed small (seconds against a generation cost measured in hours). What remains a *projection* is the generative $\bar{T}_{\text{shot}}$: the video generator is a Wan-class MoE port in progress [18], and at an estimated 90–120 s per clip a three-hour film projects to $\approx 54\text{--}72\text{ h}$ serial on one GPU versus $\approx 18\text{--}24\text{ h}$ across the measured three-node ring [2] — linear in nodes per Eq. (2), and labelled projection until the port lands a first-party per-clip number. The other substrates the estimate rests on are measured: the cross-vendor ring [2], decode parity across backends [5], and the native dub’s 11.6 fps on a VAE-on-the-critical-path pipeline (f16, GB10) [1] that is a shot-shaped workload in miniature. The systems claim is the *shape* of Eqs. (1)–(2); the assembly shape is now measured, and the generative term is the only projection left in it.

What the shape says. Three consequences are worth stating plainly. A feature render is *scalable* without algorithmic change: add nodes, render faster, linearly, until you run out of shots — there is no coherence penalty for parallelism because coherence was already externalized into the bible and the plan. It is *resumable and incremental* by construction (idempotent keys), so it behaves like a build, not a monolithic training run. And it is *cheap on the wire*, because the shot is the unit of transport — a finished MP4 of a few seconds — not a stream of activations, so the throughput job is bandwidth-light exactly where the interactive companions are bandwidth-bound. The systems difficulty of long-form generation is not holding the length; it is scheduling the throughput and assembling the leaves, and both are ordinary distributed-systems work over the series’ existing substrate.

5 Open problems specific to long-form media

These are the problems the *film* adds, on top of — not duplicating — the games companion’s game-specific problems [1] and the distributed companion’s eight [4]. Each is stated to make the open question precise, and cross-referenced where it touches a companion’s territory.

1. Perceptual metrics for narrative coherence. The metrics of §3 score whether characters look constant and the style holds; they say nothing about whether the assembled film is *coherent as a story* — whether cause precedes effect, whether an establishing shot and its reverse agree on geography, whether the edit obeys continuity (eyelines, the 180-degree rule, screen direction), whether the whole *means* anything. This is the film analog of the games companion’s playability metric [1] — in both, per-frame fidelity is measurable and the property that actually matters is not — but it is distinct: a game’s open axis is *interactive* (does the world respond to control?), a film’s is *narrative and editorial* (does the cut sequence read?). The open problem is a computable proxy for narrative and continuity coherence over an assembled shot sequence — plausibly a vision-language model scoring continuity across shot boundaries, or a learned editor’s-eye metric — and, as with playability, it is the objective the rest optimize against: the shot-length and drift trades below cannot be set without it.

2. The optimal shot-length-versus-drift trade. Shot length is a free variable with two-sided cost. Longer shots mean fewer cuts (a specific directorial feel) and fewer independent jobs, but push each shot deeper into the chunk-autoregressive drift regime (§2.3), costing intra-shot quality; shorter shots reset drift more often and parallelize better, but a film cut every two seconds is a specific, often wrong, aesthetic, and more shots mean more cross-shot identity/style variance surface (§3). There is an optimization here — choose each shot’s length to trade intra-shot drift against cross-shot variance and cut rate under a narrative-coherence constraint (problem 1) — and it couples to the systems layer, since shot length sets both N_{shots} and $\bar{T}_{\text{shot}}$ in Eq. (2). It is genuinely open because its objective (problem 1) is open and its drift term is model-specific (it depends on how fast a given generator drifts, a measured quantity per model [11, 12]).

3. Motif persistence across musical cues. The score layer (§2.5) needs a character’s theme to recur *recognizably* across cues that are generated independently and orchestrated differently — the musical instance of cross-segment identity (§2.2), but harder, because musical identity is a motif (a melodic/harmonic gesture) that must survive re-instrumentation, tempo change, and mood, not a fixed embedding to be re-injected verbatim. The open problem is a motif-conditioning mechanism for generative music [22] that preserves thematic identity under transformation, and a metric for whether two cues “contain the same theme” — neither of which the per-cue assembly of §2.5 supplies on its own. It is the audio dual of narrative coherence: assembly places the cues, but whether they cohere *musically* across a film is unquantified.

4. Lip-sync integration for dialogue shots. A dialogue shot must satisfy *two* generators at once: the video layer renders the speaking character (from the bible, §2.2) and the audio layer renders the line (per-line TTS, §2.5), and the character’s mouth must match the audio. The series has the measured piece — the native localization dub achieves lip-sync confidence LSE-C 7.05 with speaker cosine 1.0 [1] — but that pipeline dubs *existing* footage; the open problem is integrating lip-sync into *generated* dialogue shots, where video and audio are *both* being synthesized and must be made consistent. Two architectures are open and untested at film scale: generate the audio first and condition the video generator on it (audio-driven talking-head generation as the shot renderer), or generate the video and apply the measured dub as a post-pass per shot. Which composes better with the bible conditioning and the throughput schedule (§4), and at what added $\bar{T}_{\text{shot}}$, is unmeasured; the LSE-C 7.05 result is the anchor it would build on, not the answer.

5. Rights and provenance of persistent generated identities. A film bible *is* a persistent synthetic identity — a face, a voice, a name, reused across a feature and, plausibly, across sequels and a franchise. That raises questions the games companion’s per-asset provenance [1] frames but does not answer for a *persistent, reused* identity: if a character’s face is derived from a training distribution or a reference actor, what are the rights in the resulting persistent identity; who owns a generated voice reused across films; how is consent for a likeness recorded and enforced across a bible’s lifetime; and how does a viewer or a rights-holder *verify* which model and which references produced a given character. The mechanism the systems layer already provides is the hook: idempotent shot keys (§4) record, per shot, exactly which bible entities, weights, and prompts produced it, so a signed provenance manifest (C2PA-style [24]) can bind to the *identity* and travel with it across the franchise. The open problem is the rights and consent *model* over persistent generated identities, and the enforcement of it through the provenance the render already captures — a governance question the architecture is well placed to serve but does not settle.

6 Landscape: closed feature systems and the open long-video frontier

Table 2 places the paper against the landscape, with every license and claim verified from the source rather than asserted, and undisclosed figures marked n/d. The frame to read it in is this paper’s thesis: the frontier is racing to *lengthen the generator*, and this paper argues the length should instead be achieved by *assembly*, so the table’s axis of interest is not who generates the longest clip but which systems externalize coherence into structure versus internalize it into a bigger model.

The closed reference points. Meta’s Movie Gen is the most complete published feature-oriented system — a cast of media foundation models up to 30B parameters generating 1080p video with synchronized audio [15] — and it is closed: no weights, no code, a reference point the open field is measured against rather than built on, exactly as GameNGen anchors the games companion’s landscape [1]. The general-purpose closed video generators — Google’s Veo line and OpenAI’s Sora [17] — likewise ship as products, not artifacts, and neither is a feature-*assembly* system: they generate high-quality short clips, and the orchestration of clips into a film is left to the user. The closed frontier’s bet is the scaling bet: larger models, longer clips.

The open long-video frontier is a forcing-and-anti-drift literature. The open releases we build on are precisely the short-horizon generators and the techniques that extend them, and their center of gravity is the chunk-autoregressive drift problem (§2.3). Diffusion Forcing established per-token/per-chunk conditioning for sequence generation [7]; Self Forcing closed the train–test gap for autoregressive video diffusion [8]; Causal Forcing and One-Forcing drove it toward real-time and one-step generation [9, 10]; TetherCache, FramePack, and RAD attack the accumulated drift that limits how long a single continuous clip can hold [11, 12, 13]; and WorldPlay pushes long-term geometric consistency for interactive sequences [14]. SkyReels-V2 is the release that most directly targets *film*, an “infinite-length film generative model” built on a diffusion-forcing framework and released under CC-BY-4.0 [16], and it is the sharpest illustration of the two philosophies: it pursues length *inside* the generator (a longer continuous synthesis), where this paper pursues length *outside* it (assembly of short shots), and the two are complementary — a better short-shot generator makes every leaf of our assembly better. The systems reading of the table is the paper’s thesis restated: all these generators are the games companion’s fixed-shape frame loop at clip scale [1], our stack loads whichever of them is best per shot, and the feature-length coherence they individually struggle to reach is supplied *around* them by the coherence stack of §2.

7 Conclusion

Feature-length generation is read as a scaling problem — grow the generator until it holds three hours — and it is instead an *assembly* problem. A film is ~ 2000 shots of a few seconds [23], coherence across it decomposes into five layers each maintained by a small structured object and a conditioning mechanism (§2), and the long-horizon state that must persist is not a context window of pixels but a $\sim 40k$ -token screenplay and a film bible that fit, trivially, in one LLM context — so the long-horizon engine is a language model, and the video, speech, and music models are short-horizon leaves. The observation that makes it work is that film grammar makes the cut a *free reset*: the chunk-autoregressive drift the open literature fights over minute-long clips is bounded, in assembled film, to a single shot, so the games companion’s persistence-beyond-the-horizon problem

System	Origin	Scale	License	Length approach / notes
Movie Gen	Meta	30B	closed (no release)	feature-oriented, synced audio; <i>reference point</i> [15]
Veo / Sora	Google DeepMind / OpenAI	n/d	closed (product)	short-clip generators; assembly left to user [17]
SkyReels-V2	SkyReels team	n/d	CC-BY-4.0	“infinite-length film” via diffusion forcing (length <i>inside</i>) [16]
Self / Causal / One-Forcing	academic [8, 9, 10]	clip-scale	open	AR video, train-test gap, real-time, one-step
TetherCache / FramePack / RAD	academic [11, 12, 13]	clip-scale	open	anti-drift for long continuous clips
WorldPlay	academic [14]	clip-scale	open (n/d)	long-term geometric consistency, interactive
This paper	Hanzo AI	orchestration—		length <i>outside</i> the generator: hierarchical assembly (§2)

Table 2: The long-form landscape, licenses and claims verified from each source (arXiv abstract, model card, or product page); undisclosed figures marked n/d. The closed frontier (Movie Gen, Veo, Sora) and the open forcing/ anti-drift literature both pursue length *inside* the generator; this paper pursues length *outside* it, by assembly, and consumes any of these as its per-shot leaf. The two philosophies are complementary: a better short-shot generator improves every leaf of the assembly.

is dissolved rather than solved, and error accumulation splits into a real but shot-bounded intra-shot drift and a cross-shot *dispersion* around fixed anchors that does not grow with length. The systems consequence is that a feature render is embarrassingly parallel — independent shot-jobs over the measured heterogeneous ring [2], with the bible as a shared prefix that prefix-caches across the job and idempotent content-hashed shots that make the render a resumable build DAG — with near-linear speedup in nodes and a wall-clock that follows Eq. (2) for any per-shot time. We labelled that per-shot time, and every feature-length number, a projection, because no three-hour render is measured on our stack; we leaned only on measured substrate — the ring, decode parity, the native dub’s cosine 1.0 and LSE-C 7.05 — and cited it to its hardware. And we set the film-specific open problems on top of the companions’ agendas rather than duplicating them: perceptual narrative-coherence metrics, the shot-length/drift trade, musical-motif persistence, lip-sync for generated dialogue, and the rights and provenance of persistent synthetic identities. The through- line is one sentence: long-form media is not generated long, it is *assembled* long, and the intelligence lives in the plan, the bible, and the schedule — not in a model that holds three hours it never needs to hold.

References

- [1] Hanzo AI — ML Systems. *The Frame Is the Token: Generative Videogames on a Native Inference Stack*. Games crossover in the native-inference series.
- [2] Hanzo AI — ML Systems. *One Cluster, Every Vendor: Cross-Vendor Tensor-Parallel LLM Inference over Commodity Ethernet*. Distributed-inference companion (measured).
- [3] Hanzo AI — ML Systems. *The Latency Physics of Geo-Distributed Autoregressive Decode*. Distributed-inference companion (analytical).
- [4] Hanzo AI — ML Systems. *Open Problems in Heterogeneous Distributed Inference*. Distributed-inference companion (research agenda).
- [5] Hanzo AI. *Edge LLM Inference Across Commodity Accelerators: hanzo-engine at llama.cpp Decode Parity on Every Backend*. Paper 05 (measured).
- [6] Hanzo AI — ML Systems. *hanzo-film: A Native Plan-to-Timeline Orchestrator for Feature-Length Assembly*. Engine component (measured; merged at `cdca071b4`).
- [7] B. Chen, D. Martí Monso, Y. Du, M. Simchowitz, R. Tedrake, V. Sitzmann. *Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion*. arXiv:2407.01392.
- [8] X. Huang, Z. Li, G. He, M. Zhou, E. Shechtman. *Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion*. arXiv:2506.08009.
- [9] H. Zhu, M. Zhao, G. He, H. Su, C. Li, J. Zhu. *Causal Forcing: Autoregressive Diffusion Distillation Done Right for High-Quality Real-Time Interactive Video Generation*. arXiv:2602.02214.
- [10] J. Feng, J. Cui, Y. Ban, C.-J. Hsieh. *One-Forcing: Towards Stable One-Step Autoregressive Video Generation*. arXiv:2605.23458.
- [11] Y. Meng, X. Luo, L. Li, W. Jiang, C. Gao, X. Chen, Y. Li, X.-P. Zhang. *TetherCache: Stabilizing Autoregressive Long-Form Video Generation with Gated Recall and Trusted Alignment*. arXiv:2606.13035.
- [12] L. Zhang, S. Cai, M. Li, G. Wetzstein, M. Agrawala. *Frame Context Packing and Drift Prevention in Next-Frame-Prediction Video Diffusion Models (FramePack)*. arXiv:2504.12626.
- [13] T. Chen, Z. Ding, A. Li, C. Zhang, Z. Xiao, Y. Wang, C. Jin. *Recurrent Autoregressive Diffusion: Global Memory Meets Local Attention*. arXiv:2511.12940.
- [14] W. Sun, H. Zhang, H. Wang, J. Wu, Z. Wang, Z. Wang, Y. Wang, J. Zhang, T. Wang, C. Guo. *WorldPlay: Towards Long-Term Geometric Consistency for Real-Time Interactive World Modeling*. arXiv:2512.14614.
- [15] A. Polyak, A. Zohar, A. Brown, et al. (Meta). *Movie Gen: A Cast of Media Foundation Models*. arXiv:2410.13720.
- [16] G. Chen, D. Lin, J. Yang, et al. *SkyReels-V2: Infinite-length Film Generative Model*. arXiv:2504.13074. CC-BY-4.0.
- [17] OpenAI. *Video Generation Models as World Simulators (Sora)*. Technical report, 2024. (Closed; product.)
- [18] Wan Team, Alibaba Tongyi Lab. *Wan: Open and Advanced Large-Scale Video Generative Models (Wan 2.2)*. Apache-2.0.
- [19] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685.

- [20] N. Ruiz, Y. Li, V. Jampani, Y. Pritch, M. Rubinstein, K. Aberman. *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. CVPR, 2023.
- [21] H. Ye, J. Zhang, S. Liu, X. Han, W. Yang. *IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models*. arXiv:2308.06721.
- [22] ACE Studio and StepFun. *ACE-Step: A Step Towards Music Generation Foundation Model*. Apache-2.0.
- [23] J. E. Cutting, J. E. DeLong, C. E. Nothelfer. *Attention and the Evolution of Hollywood Film*. Psychological Science 21(3):440–447, 2010.
- [24] Coalition for Content Provenance and Authenticity. *C2PA Technical Specification*. Content provenance standard.